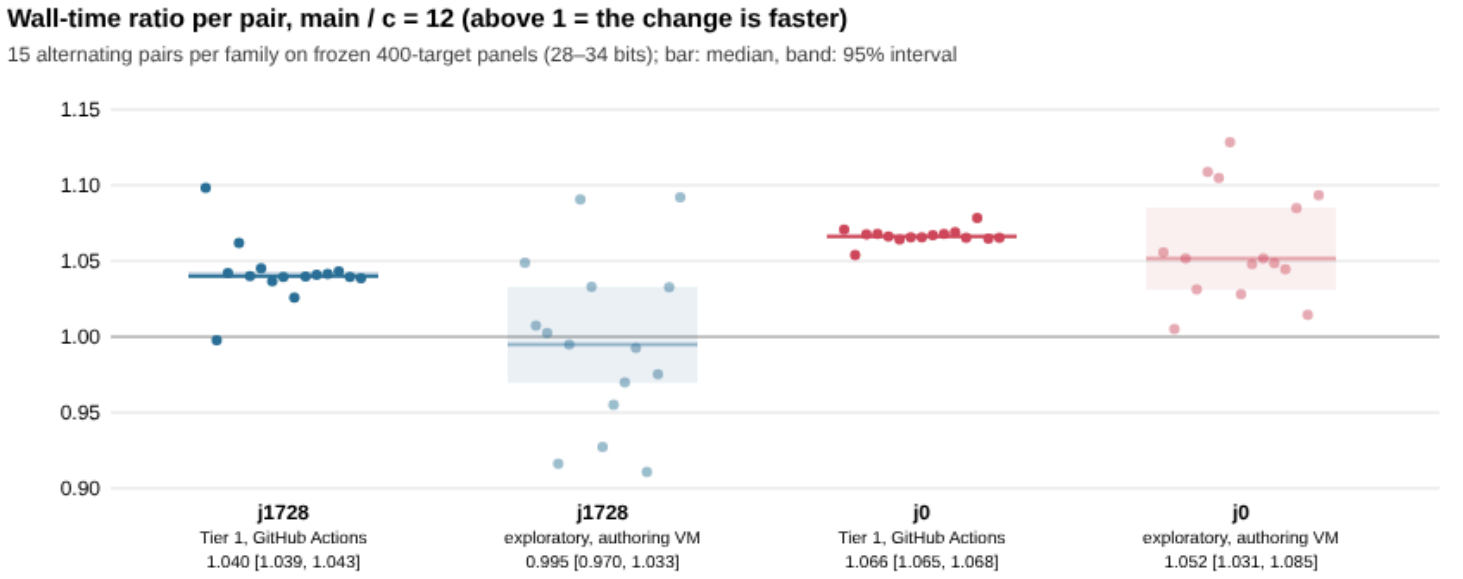
Wall time of the GLV table rule c = 12: Tier 1 speedup on both families

Result. On a GitHub Actions `ubuntu-24.04` runner, under the repository's Tier 1 gate, #1065's change (table budget constant 48 → 12, merged as cfc31eb) makes `curve_solve` faster in wall time on both frozen panels. The runner populated `paired_speedup` for both families: a qualified Tier 1 speedup, scoped to this instance.

family	wall-time ratio main / c = 12	95% paired bootstrap	valid pairs	CPU-time ratio	operation ratio on this panel
j = 1728	**1.040**	[1.039, 1.043]	15 / 15	1.040	1.018
j = 0	**1.066**	[1.065, 1.068]	15 / 15	1.066	1.033

- **Provider:** GitHub Actions.
- **Instance:** `gh-40039118727-1-paired-walltime` (run 40039118727).
- **Host:** standard `ubuntu-24.04` runner, image 20261012.334.1, AMD EPYC 7763 with 2 exposed CPUs, cgroup v2 with no CPU quota.
- **Execution:** both arms pinned to CPU 0, alternating r c c r … over 15 pairs per family.
- **Gate:** every pair passed the runner's gates, which reject a pair on a throttle, OOM, steal-time increment, major fault, failed verification, timeout, or changed input, source or binary. `binaries_stable`, `artifacts_stable` and `runner_stable` were all true.
- **Limits:** `host_exclusivity_proved` is false, as it always is for Tier 1. This is an instance-scoped result, not a host-isolated (Tier 0) one.
- **Binaries:** the runner rebuilt both arms from the pinned commits. Reference is 7cb3dce; candidate is f630cc3, which is 7cb3dce plus #1065's `src/curve.c`. The rebuilt binaries are byte-identical to the frozen hashes in `PROTOCOL.md` (`32630811…`, `43a89384…`). Every run printed `verified=1` with the frozen hash of the true scalars.
- **Receipts:** `tier1-github-40039118727/` holds the allocation receipt, host probe, preflight, manifests, run summaries, the raw `runs.jsonl` and `pairs.jsonl`, and every run's stdout and stderr. The 6.5 MB binaries are not committed; `binaries.sha256` lists their hashes.



Why wall time gains more than the operation count (interpretation, not measured)

The panel's operation saving is 1.8% (j = 1728) and 3.3% (j = 0). Wall and CPU time both fall about twice as much. `group_ops` counts every group operation as one, but they do not all cost the same:

- **Escape doubling:** the cycle escape's doubling is an affine operation outside the walk's batched inversion (#953, limits). With an 8-entry table the walk escapes about 25× more often than with 32 entries (#1059: 17.9 vs 0.68 per 10^4 steps for j = 1728 at 28 bits).
- **Retries:** look-ahead retries also fall with the larger table.

If those operations cost more than an average batched step, operation counts understate the gain. This explanation has not been measured. A per-operation cycle profile on a host with a PMU would test it (the `perf-ipc` skill).

How the run got here (all recorded in `PROTOCOL-AMENDMENT-1.md`)

1. **Runpod pod rejected (no timing).** The pod rented under `PROTOCOL.md` (`srwxamrnnf2mye`, flavor `cpu5c`) failed the Tier 1 preflight: it runs cgroup v1. Its receipts are in `runpod-rejected-20261023/`.
2. **Host amended.** Amendment 1 moved the run to a GitHub-hosted runner, the repository's existing Tier 1 path, before any timing.
3. **Two workflow attempts failed before any pair ran.**
4. Run 40030129318 (Fix 1): a `$GITHUB_ENV` variable was read in the step that set it.
5. Run 40036000239 (Fix 2): the runner ran out of disk on full worktrees. The fix is sparse, blob-less checkouts.

Neither attempt measured anything.

Earlier exploratory run (untiered, superseded)

The authoring VM (4-vCPU KVM, cgroup v1, nonzero steal) ran the same binaries and panels with `exploratory_local.py`:

- **j = 1728:** 0.995 [0.970, 1.033].
- **j = 0:** 1.052 [1.031, 1.085].

On that host the j = 1728 gain was hidden in timing noise. The rows are in `rows_exploratory_local.jsonl`, the summary in `summary_exploratory_local.json`, and the host manifest in `host_exploratory_local.json` and `lscpu_exploratory_local.txt`.

Limits

- **Panels:** two frozen 400-target panels, 28–34-bit subgroups only. Each sums the operations of 100 targets per size, so 34-bit curves dominate. There the change saves least (held-out 0.994 / 0.980). At 26 bits and from 36 bits up, the change picks the same table as before.
- **Hosts:** one runner instance and one CPU model (AMD EPYC 7763).